Navigating the Privacy–Robustness–Performance Trilemma in Federated Learning

Jonas Müller
7399328
jonas.mueller-1@studium.uni-hamburg.de

Rebekka Schnoor
7159351
rebekka.schnoor@studium.uni-hamburg.de

July 13, 2026

[2–4 sentences summarizing the paper: motivation (the trilemma), what we did (8-configuration empirical study on MNIST/CIFAR-10 with Flower+Opacus+BLADES), key finding (compounding behavior, direction of magnitude), and main take-away for practitioners.]

Contents

List of abbreviations

FL Federated Learning

IID independent and identically distributed

DP-SGD Differentially Private Stochastic Gradient Descent

SGD Stochastic Gradient Descend

DP Differential Privacy

1 Introduction

We present a systematic empirical study of the Privacy–Robustness–Performance trilemma in Federated Learning (FL), examining how simultaneously deployed defenses interact and compound across all three axes.

[Summarize results and contributions.]

1.1 Background and Motivation

In the past ten years, FL has emerged as a framework for achieving distributed training without the need for data centralization. This is of particular interest when processing sensitive data, such as in healthcare or mobile applications. The local training allows to leave the data with the clients but opens new attack surfaces for model poisoning during training through the transmission of manipulated updates. Also, the transmitted gradients can still leak information about the used training data set. Additionally, gradient transmission at each round causes significant communication cost, especially for complex models.

In an effort to fortify the resilience of models and safeguard client privacy, numerous methodologies have been implemented in recent years. Robustness-enhancing methods are designed to detect malicious updates by assuming a significant deviation from the honest majority. Privacy-preserving algorithms add a mask or calibrated noise to the local updates before transmitting them to the central server.

1.2 Problem Statement

Ensuring model robustness and client privacy while balancing communication cost and model accuracy has been identified as a significant challenge. Adding privacy-enhancing noise to individual updates reduces a malicious server's ability to extract information about local datasets, while simultaneously reducing model accuracy. Discarding unusual updates improves robustness against Byzantine adversaries but risks disregarding legitimate updates, particularly in settings where data is not independent and identically distributed (IID). Compression techniques that reduce transmission load limit the information transmitted to the server and thus reduce model accuracy.

In 2023, Allouah et al. [1] proved Theorem 1, stating that the combination of such methods yields a lower bound for the expected training error that exceeds the sum of the individual cost for robustness and privacy by a multiplicative term.

Theorem 1. *Consider distributed mean estimation in $\mathbb{R}^d$ with n users, of which an η-fraction may behave maliciously. Any algorithm, producing an estimate $\hat{\mu}$ of the true mean $\mu \in \mathbb{R}^d$ of inputs, and achieving (ε, δ)-local DP and robustness to η-corruption must incur an estimated error*

$$\mathbb{E} \parallel \hat{\mu} - \mu \parallel_2^2 = \tilde{\Omega}\left(\frac{\eta}{\varepsilon^2} + \frac{d}{\varepsilon^2 n} + \eta G^2\right). \tag{1}$$

A position paper by the same group [2] highlights a gap between the theoretical lower bound and current implementations.

1.3 Research Questions

The project is structured around three research questions:

1. **Isolation:** How do differential privacy, Byzantine-robust aggregation, and gradient compression individually affect model accuracy, robustness, and computational efficiency?
2. **Combination:** How do pairwise and full combinations of the three mechanisms interact? Do the costs compound additively, sub-additively, or super-additively?
3. **Trust and Privacy:** How do the trust scores assigned by FLTrust evolve throughout training, and what do they reveal about the interaction between privacy, robustness, and model performance?

1.4 Contributions

[Bullet list mirroring Section 6 of the Exposé: (i) systematic 8-configuration mapping on MNIST/CIFAR-10 with joint trilemma reporting; (ii) reproduction and cross-framework extension of BLADES baselines; (iii) initial test of joint clipping-norm calibration as a mitigation heuristic; (iv) open-source reproducible pipeline.]

1.5 Structure of the Report

The remainder of the report is organized as follows. Section 2 provides the theoretical background of all three trilemma axes and derives the joint lower bound. Section 3 defines the threat model and evaluation metrics. Section 4 describes the chosen mechanisms and their parametrization. Section 5 details the experimental setup and software stack. Section 6 presents and analyses the empirical results. Section 7 interprets findings, proposes mitigation, and tests the joint calibration heuristic. Section 8 addresses limitations and future work. Section 9 concludes.

2 Background

This chapter summarizes the theoretical background of FL, the trilemma axes and formalizes the Privacy-Robustness-Performance trilemma.

2.1 Federated Learning

Federated Learning is a machine learning framework comprising a central server and multiple distributed clients. Each client maintains a local dataset to compute updates for the global model. The central server orchestrates the training process by distributing the current global model, collecting client updates, and aggregating them to produce a new global model update. This update is then distributed to clients to initiate the next training round.

The most widely used aggregation method is FedAvg [3], which computes the global update as a weighted average of the client-local model updates.

2.2 Privacy: DP

This chapter introduces the concepts of local and central DP and describes the implementation of local DP in DP-SGD.

2.2.1 Definition and Composition

DP forms a good standard for privacy guarantees for algorithms used in data aggregation. Its degree of protection is defined by the privacy budget ε and the failure probability δ. We use the definition given by Abadi et al. [4].

Definition 1. *A randomized mechanism $M\colon D \to R$ with domain D and range R satisfies (ε, δ)-differential privacy if for any two adjacent inputs $d, d' \in D$ and for any subset of outputs $S \subseteq R$ it holds that*

$$\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta.$$

The privacy budget ε denotes the trade-off between utility and privacy, bounding the tolerated noise addition. Small values ($\varepsilon < 1$) provide the strongest privacy guarantees while larger values ($\varepsilon > 8$) allow for more accurate training.

The failure probability δ, introduced by Dwork et al. [5], denotes the probability that the plain ε-DP is broken and should be chosen smaller than the inverse size of the local dataset.

Algorithm 1 Differentially private SGD (Outline)

Input: Examples $\{x_1, \ldots, x_N\}$, loss function $\mathcal{L}(\theta) = \frac{1}{N}\sum_i \mathcal{L}(\theta, x_i)$. Parameters: learning rate η_t, noise scale σ, group size L, gradient norm bound C.

Initialize θ_0 randomly

for $t \in [T]$ **do**

Take a random sample L_t with sampling probability L/N

Compute gradient

For each $i \in L_t$, compute $\mathbf{g}_t(x_i) \leftarrow \nabla_{\theta_t}\mathcal{L}(\theta_t, x_i)$

Clip gradient

$\bar{\mathbf{g}}_t(x_i) \leftarrow \mathbf{g}_t(x_i) / \max\left(1, \frac{\|\mathbf{g}_t(x_i)\|_2}{C}\right)$

Add noise

$\tilde{\mathbf{g}}_t \leftarrow \frac{1}{L}\left(\sum_i \bar{\mathbf{g}}_t(x_i) + \mathcal{N}(0, \sigma^2 C^2 \mathbf{I})\right)$

Descent

$\theta_{t+1} \leftarrow \theta_t - \eta_t \tilde{\mathbf{g}}_t$

Output θ_T and compute the overall privacy cost (ε, δ) using a privacy accounting method.

Figure 1: DP-SGD pseudocode as published by Abadi et al. [4].

2.2.2 DP-SGD

DP-SGD is a modification to the standard Stochastic Gradient Descend (SGD) algorithm introduced by Abadi et al. [4] in 2016 to integrate DP into deep learning frameworks. It enhances SGD by clipping the computed gradient to a fixed norm and adding calibrated (Gaussian) noise before performing an update step. The algorithm pseudocode as proposed by Abadi et al. is shown in figure 1.

2.2.3 Central vs. Local DP in FL

In central DP, the server adds noise after aggregating the client updates. This setting assumes a trusted aggregator while protecting the aggregated updates before they are shared with third parties.

In the considered setting, client updates must be protected against inference attacks by the aggregator or adversaries eavesdropping on the client–server communication, requiring local DP. To achieve this, each client adds noise to its update before transmission. Since the noise added by individual clients accumulates during aggregation, local DP requires a higher effective noise magnitude and therefore incurs a greater loss in accuracy.

2.3 Robustness: Byzantine-Tolerant Aggregation

This section introduces the Byzantine fault model and presents two aggregation schemes designed to mitigate the effects of Byzantine adversaries.

2.3.1 Byzantine Fault Model

Byzantine Fault Tolerance is a property of a distributed system that allows for the system to stay operational while some components fail or act maliciously.

A η-fraction Byzantine adversary means that $\eta\%$ of the clients are acting maliciously. In the conducted experiments, malicious behavior means communicating manipulated gradients to poison the central model, e.g. insert a backdoor, degrade the general performance or enable label flipping attacks.

2.3.2 Krum

Krum [6], proposed by Blanchard et al. in 2017, enhances robustness to Byzantine adversaries by selecting updates closest to their XX neighbors, effectively focusing on the densest cluster of updates of size XX. This ensures the training process is provably Byzantine-tolerant but relies on the assumption that honest updates cluster together, which only holds when data distributions across clients are similar. Under data heterogeneity, this assumption breaks down, leading to performance degradation because honest updates may be discarded if they deviate significantly from the majority.

2.3.3 FLTrust

FLTrust [7] was introduced by Cao et al. in 2020 and mitigates malicious updates through trust scores. These are obtained by calculating the cosine similarity of the local update to an update anchor computed on a small root dataset held by the server. The method relies on the assumption that the server has access to a dataset representing the general data distribution across all clients.

2.4 Performance: Communication Efficiency

Another practical limitation especially in mobile applications is the available bandwidth to transmit local updates to the aggregator. We therefore include a method to communicate sparse instead of dense updates.

2.4.1 Top-k Gradient Sparsification

Top-k gradient sparsification [8] reduces communication during gradient transmission by retaining only the k largest gradient components, where k is a fixed fraction of the model dimension d.

Discarding the remaining gradient components inevitably results in information loss, leading to reduced model performance. The extent of this degradation depends on how the optimization information is distributed across the gradient, with models relying on a small number of dominant components generally being less affected than those requiring information from many dimensions.

2.4.2 Relation to Model Dimensionality

The term $d/(\varepsilon^2 n)$ in the theoretical lower bound [1] raises the assumption that //TODO

2.5 The Privacy–Robustness–Performance Trilemma

This section first presents the theoretical lower bound of the trilemma and then discusses its implications for empirical evaluation, motivating efficiency as a fundamental third axis.

2.5.1 Theoretical Lower Bound

Allouah et al. [1] proved that any algorithm simultaneously guaranteeing ε-local DP and robustness against an η-fraction Byzantine adversary incurs a mean estimation error of at least

$$\Omega! \left(\frac{\eta}{\varepsilon^2} + \frac{d}{\varepsilon^2 n} + \eta G^2 \right). \tag{2}$$

This lower bound establishes that the costs of privacy and Byzantine robustness are fundamentally coupled. In particular, the cross-term η/ε^2 shows that these costs interact multiplicatively rather than additively.

2.5.2 Efficiency Axis Extension

The same authors proposed the aggregation schemes SMEA and CAF, which match this theoretical lower bound. However, both methods incur a runtime complexity that scales cubically with the model dimension, rendering them impractical for large-scale models. Furthermore, the runtime of SMEA grows exponentially with the number of tolerated Byzantine adversaries, making it infeasible in realistic federated learning settings.

The term $\frac{d}{\varepsilon^2 n}$ in 2 further relates the model dimension d to the communication cost and achievable estimation error. Consequently, maintaining accuracy for high-dimensional models requires either relaxing privacy guarantees or integrating a larger number of participating clients.

2.5.3 Empirical Evidence of Compounding

In 2021, Guerraoui et al. empirically investigated the question *"Differential Privacy and Byzantine Resilience in SGD: Do They Add Up?"* [9]. They showed that a naive combination of privacy- and robustness-enhancing methods, namely DP-SGD and Minimum-Diameter Averaging (MDA) [10], leads to unfavorable scaling with the model dimensionality and super-additive performance degradation. These findings support the theoretical results of Allouah et al. and reinforce the need to treat efficiency as a fundamental third axis in future experimental studies.

3 Threat Model and Metrics

This chapter maps out the assumed threat model and introduces the metrics used to evaluate the three axes.

3.1 Threat Model

We assume an honest-but-curious central server that adheres to protocol specifications but may attempt to infer private information from transmitted updates, motivating the use of differential privacy (DP) to safeguard client data. The adversary is characterized by a fraction η of malicious clients, capable of arbitrarily manipulating gradients (e.g., through label-flipping attacks, as formalized in Fang et al. [11]). These clients aim to corrupt the global model by injecting adversarial updates. Crucially, we assume no collusion between Byzantine clients and the server, restricting adversarial behavior to local manipulations of gradients. This assumption simplifies the analysis while maintaining practical relevance for decentralized learning systems.

3.2 Evaluation Metrics

As suggested by Allouah et al. [2], every configuration is evaluated simultaneously across all three trilemma axes:

Privacy. The privacy objective is quantified by the assigned (ε, δ)-differential privacy (DP) budget.

Robustness. Robustness is evaluated by the the accuracy gap Δ_{acc} relative to a clean FedAvg baseline. The accuracy gap captures performance degradation caused by adversarial clients, providing a measure of resilience to Byzantine behavior.

Performance. The performance metric includes test accuracy on held-out data, communication cost (bytes sent per client per round), and wall-clock training time. These metrics collectively assess the trade-off between model utility, resource efficiency, and computational feasibility in decentralized learning systems.

3.3 Trilemma Visualization

[Radar / simplex chart positioning each of the 8 configurations relative to the three axes, analogous to Figure 1 of [2]. Include actual figure once results are available.]

4 Methods

This section briefly describes and justifies the selected methods for each axis.

4.1 Baseline: FedAvg

[Standard weighted average aggregation [3]; serves as the reference point for all accuracy gaps; no privacy, robustness, or compression applied.]

FedAvg, introduced by McMahan et al. in 2017 [3], is a widely used aggregation strategy in FL designed to handle non-IID data efficiently. It computes a weighted average of local model updates, with weights determined by the size of each client's local dataset. The paper establishes baselines on standard datasets such as MNIST and CIFAR-10, making FedAvg a foundational reference for evaluating FL methods. Its simplicity and effectiveness in early FL research have cemented its role as a standard baseline for comparative studies.

4.2 Privacy Mechanism: DP-SGD

[Per-sample clipping norm C; noise multiplier σ; privacy budget $\varepsilon \in \{1, 5, 10\}$, $\delta = 10^{-5}$;]

4.3 Robustness Mechanism: FLTrust

[Server root dataset construction; trust score computation; cosine similarity clipping; parameter: root dataset size;]

The root dataset for the central server was obtained by randomly sampling a parametrized number of data points, uniformly distributed over all classes, from the training set.

The trust score for weighting the local contributions is calculated as the cosine similarity to the update vector that was computed from the root data set as depicted in figure 2.

4.4 Performance Mechanism: Top-k Sparsification

[Fraction $k \in \{1\%, 10\%\}$ of d transmitted per round; error feedback accumulator; custom Flower strategy wrapper; interaction with gradient clipping discussed in Section 4.5.]

4.5 Mechanism Interaction Effects

[Theoretical analysis of how clipping (DP) and cosine-similarity clipping (FLTrust) may amplify each other; how sparsification changes the effective gradient distribution seen by the aggregator; hypothesis: super-additive cost for the full combination.]

5 Experimental Setup

5.1 Experimental Grid

The experimental grid is described in table 1.

Identifier	DP-SGD	FLTrust	Top-k
Base			
DP	x		
FLTrust		x	
TOPK			x
DP+FLTrust	x	x	
DP+TOPK	x		x
FLTrust+TOPK		x	x
ALL	x	x	x

Table 1: Identifiers and applied methods for the different experimental setups.

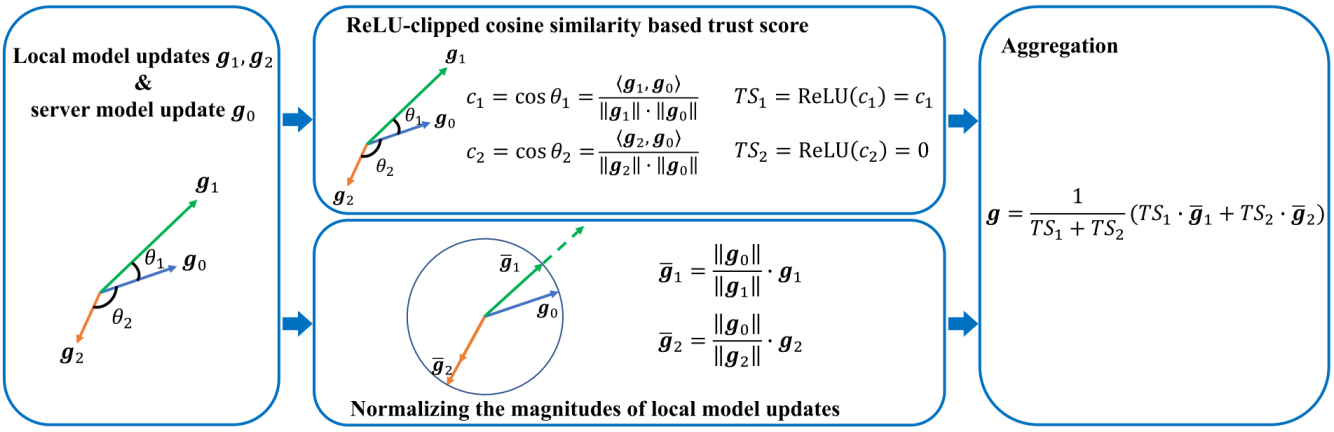

Figure 2: Global aggregation rule for FLTrust as introduced by Cao et al. [7].

5.2 Datasets

MNIST. The MNIST dataset consists of 70,000 grayscale images of handwritten digits with a resolution of 28 × 28 pixels, each labelled with the corresponding digit in the range [0,9]. It is divided into a training set of 60,000 samples and a test set of 10,000 samples. Owing to its simplicity, the dataset is widely used to benchmark image classification models and verify their functionality before extending them to more complex datasets.

CIFAR-10. CIFAR-10 is a benchmark dataset for object classification comprising 60,000 RGB images with a resolution of 32 × 32 pixels, categorized into 10 classes. It is divided into 50,000 training samples and 10,000 test samples and is widely used for training and benchmarking object classification networks.

5.3 Learning and attack settings

For a training setup with n clients, the dataset is partitioned into $n+1$ disjoint subsets: one uniformly sampled root dataset for FLTrust and n randomly sampled client datasets. We assume an IID data distribution across all clients and therefore do not introduce artificial label imbalances. The resulting label distribution for the MNIST experiments is shown in Figure 5.

The label-flipping attack is implemented by swapping the labels of the digits "3" and "7" in the MNIST dataset and the classes "cats" and "dogs" in the CIFAR-10 dataset for the malicious clients. The resulting gradients are subsequently scaled by the attack scale parameter before being transmitted to the central server to amplify the attack.

5.4 Parameters

[review]

Static. The following parameters are constant across all experiment runs.

- The client learning rate is fixed at 0.01.
- The initial seed for random processes is fixed to 42 to ensure comparability across runs. Randomizers required to vary between rounds, like training set shuffle or DP noise generation, additionally take the round number as input.
- The attack type is fixed to flip specific labels: Digits 3 and 7 on the MNIST dataset and TODO: CIFAR10
- DP failure probability δ is fixed to 1e-5.

Dynamic. The following parameters are varied across experiment runs to investigate their influence on the results.

- Number of clients
- Number of training rounds
- Byzantine fraction for label flipping attack
- Attack scale for label flipping attack
- Root dataset size for FLTrust
- DP budget $\varepsilon \in [1; 5; 10]$
- Top-K sparsification threshold $k \in [1\%; 10\%; 50\%]$

5.5 Software Stack

- **Flower** (flwr) [12] — FL orchestration, client/server simulation, strategy interface.
- **Opacus** [13] — DP-SGD, Rényi DP accounting, gradient clipping.
- **FLTrust** [7] — FLTrust defense implementation.
- **Custom** — Top-k gradient sparsification wrapper.

[Software versions; hardware (GPU/CPU); random seed management; number of independent runs per configuration.]

5.6 Reproducibility

[Pointer to open-source repository; instructions to replicate all 8 configurations; fixed random seeds; containerization (Docker/Conda environment file).]

6 Results

This section summarizes the experimental results. The final round accuracy over all configurations is reported in Figure 6.

6.1 Baseline Reproduction (RQ1 – Sanity Check)

[FedAvg and FLTrust baselines on MNIST vs. published BLADES numbers; table: our accuracy / ASR vs. reported values; discussion of any deviations.]

6.2 Isolation: Individual Mechanism Costs (RQ1)

This chapter analyzes the cost and effects of the isolated mechanisms compared to the baseline.

6.2.1 DP-SGD Cost

[review]

Cost. The replacement of pure SGD with DP-SGD on the clients results in an accuracy loss of 0.2 compared to the baseline and an increase in training time of TODO

Contrary to our expectations, the assigned privacy budget does not have a major influence on the accuracy besides the general degradation observed when applying DP-SGD: We achieve similar performance for all $\varepsilon \in [1; 5; 10; 25]$. Figure 7 shows a slight variation in the early rounds but overall a similar evolution for all values of ε.

6.2.2 FLTrust

[review, ggf charts on training time??]

Effect. Enabling FLTrust improves model accuracy and mitigates the effects of the label-flipping attack, as shown in Figure 3. In the baseline model, the targeted digits 3 and 7 are consistently misclassified as each other, whereas enabling FLTrust largely restores correct predictions. The improvement becomes more pronounced with increasing Byzantine fractions and attack scales. The results shown were obtained with a Byzantine fraction of $\eta = 40\%$ and an attack scale of 2.0.

Cost. *[extend once results are available]*

For a configuration with 50 clients and 50 training rounds, applying FLTrust increased the wall-clock training time by 29.4%. With fewer clients, the relative overhead decreased, becoming negligible for 10 clients and 5 training rounds.

Development of trust scores. Figure 9 illustrates the evolution of the trust scores throughout training. The first chart, corresponding to the isolated FLTrust configuration, shows a clear separation between honest and malicious clients. Honest clients start with trust scores of approximately 0.9, which gradually decrease after roughly 15 rounds before stabilizing around 0.3. Malicious clients start near 0.7, exhibit a brief increase, and then decline sharply to 0 after approximately 25 training rounds.

One possible explanation for the gradual decline in the trust scores of honest clients is that differences between the local training datasets become increasingly influential once the model has learned the general structure of the data. Consequently, honest updates remain broadly aligned with the root dataset but become progressively more diverse, whereas the poisoned and scaled updates of

malicious clients deviate from the server update anchor and are therefore assigned trust scores close to zero.

6.2.3 Top-k Sparsification Cost

[Accuracy, bytes per round, and wall-clock time for $k \in \{1\%, 10\%\}$. REVIEW]

Effect. Applying Top-k sparsification yields EFFECT GOES HERE

Cost. We do not observe a significant decline in performance when applying Top-k sparsification to the MNIST training process. Cutting down to transferring only the top 1% of the gradient yields a slight loss in accuracy, but not a noticeable effect.

One possible explanation for the minimal effect of the sparsification is that MNIST is a relatively simple dataset for which the classification depends on a very limited number of features.

6.3 Combination: Pairwise and Full Interactions (RQ2)

[Main results table: 8 configurations $\times$ all metrics; additive vs. super-additive cost analysis; focus on DP+FLTrust interaction; radar / simplex visualization (Figure [X]).]

This section reports the combinatory effects observed when pairing multiple mechanisms.

6.3.1 DP + Robustness

[check when results]

Cost. Combining DP-SGD with FLTrust results in a substantial decline in model accuracy, reducing performance to only slightly above random guessing. Figure 4 shows the confusion matrix of the final model, revealing that it predicts only two classes.

These results are consistent with the super-additive interaction between privacy and Byzantine robustness derived by Allouah et al. Intuitively, the noise introduced for differential privacy causes client updates to deviate from the anchor update computed on the root dataset, resulting in uniformly low trust scores. This behavior is reflected in the second chart of Figure 9.

Influence of the ε value. Although model performance varies slightly with ε, even the relaxed setting of $\varepsilon = 25$ does not achieve an accuracy above 0.2. The most pronounced effect is shown in Figure 8, which illustrates the accuracy throughout training. While configurations with $\varepsilon \in 5, 10, 25$ reach a peak accuracy of approximately 0.2 after 35, 25, and 15 training rounds, respectively, the configuration with $\varepsilon = 1$ never exceeds an accuracy of 0.12, indicating substantially slower convergence.

6.3.2 DP + Compression

[review when results available]

Applying Top-k sparsification to updates perturbed by DP noise significantly affects model performance, an effect that is not observed in the Top-k-only configuration. While $k = 50\%$ does not introduce additional degradation beyond that caused by DP-SGD alone, smaller values of k substantially reduce the final model accuracy, with $k = 1\%$ reaching a peak accuracy of only 0.2.

As in the DP-only configuration, varying the privacy budget ε has no noticeable impact on model accuracy.

6.3.3 Robustness + Compression

[review with results.]

Combining FLTrust with Top-k sparsification largely preserves the individual effects of both methods. Transmitting only 1% of the gradient results in the same slight reduction in accuracy observed in the Top-k-only configuration, while FLTrust provides a comparable improvement in robustness to that achieved with dense updates. Figure 9 shows that the trust scores exhibit a similar evolution to the dense-update setting.

6.3.4 Full Combination (DP + Robustness + Compression)

[Worst-case configuration; super-additive compounding quantified.]

Combining all three methods reduces model performance to a level comparable to the DP+FLTrust configuration. The final accuracy ranges between 0.12 and 0.18, only marginally exceeding random guessing. The resulting confusion matrix is likewise similar to that of the DP+FLTrust setting.

6.4 MNIST vs. CIFAR-10 Comparison

[How results scale with dataset complexity; does the compounding pattern hold across both?]

7 Discussion

7.1 Interpretation of Compounding Effects

[Relate empirical findings to the theoretical cross-term η/ε^2; characterize compounding as additive / sub-additive / super-additive per configuration pair; intuition for why DP clipping and FLTrust cosine clipping interact destructively.]

7.2 Mitigation: Joint Clipping-Norm Calibration (RQ3)

[Heuristic: jointly set DP clipping norm C and FLTrust trust threshold to minimize redundant gradient suppression; description of the calibration procedure; results vs. uncalibrated worst-case combination; formal guarantee preservation argument.]

7.3 Practical Recommendations

[When to prioritize which axis; safe operating region in the trilemma space; guidance on (ε, k) parameter selection given a robustness requirement.]

8 Limitations and Future Work

8.1 Limitations

[Single attack type (label-flipping); single aggregation rule (FLTrust); membership inference approximation (shadow model only); limited ε grid; MNIST/CIFAR-10 may not represent medical/financial FL.]

The experiments in this project were conducted under several simplifications to provide a lightweight anchor for future research.

Simple models and datasets. The datasets and models selected offer good comparability with existing work but may not fully represent the data analyzed in practical FL applications (e.g., financial or medical domains) or complex architectures (e.g., transformer-like models).

Limited method coverage. We evaluated only a single method and at most one attack type per axis. The baseline for membership inference attack success rate was approximated using a shadow model, and the privacy budget was constrained to a limited ε-grid to balance computational feasibility with analytical rigor.

Privacy budget schedule. The privacy budget was spent uniformly over time, but recent research by Kiani et al. [14] suggests that a time-adaptive approach yields better privacy-utility-tradeoffs.

Stateless Clients. The clients do not preserve a state over multiple rounds. In combination with top-k sparsification, this hinders small but repeated updates to accumulate to meaningful contributions over time. Introducing a client state requires well-behaved clients to correctly maintain the state and stay synchronized with the central model, which is realistic for distributed training but not a reasonable assumption in federated learning. Another option is to share the state with the central server, defeating the purpose of the sparsification and introducing new privacy vulnerabilities.

8.2 Future Work

[Extend to Krum, SMEA, CAF aggregators; stronger attacks (model poisoning [11]); LoRA+DP reparametrization [15] for the efficiency axis; theoretical tightness of joint calibration heuristic.]

Future work should aim to expand the detailed analysis to CIFAR-10 and more complex datasets.

Furthermore the experimental setup should be extended to encompass a broader range of methods (e.g., Krum, SMEA, CAF for aggregation) and evaluate stronger adversarial scenarios, such as model poisoning attacks described by Fang et al.[11]. Additionally, exploring reparametrization techniques such as the Low-Rank-adaptation method proposed by Liu et al.[15] could provide insights into balancing efficiency and privacy in FL systems.

Other promising directions include testing alternative threat models (e.g., malicious servers, collusion between malicious clients and servers), or introducing data heterogeneity through artificially distributed datasets. These extensions would better reflect real-world complexities and enable more robust comparisons across methodologies.

9 Conclusion

[Restate research questions and answers; key quantitative finding (e.g. full combination degrades accuracy by X% beyond sum of individual costs); mitigation heuristic reduces compounding by Y% without formal guarantee loss; open-source pipeline as contribution to the community.]

References

[1] Y. Allouah, R. Guerraoui, N. Gupta, R. Pinot, and J. Stephan, "On the privacy-robustness-utility trilemma in distributed learning," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, 2023. [Online]. Available: https://arxiv.org/abs/2302.04787

[2] Y. Allouah, R. Guerraoui, and J. Stephan, "Balancing privacy, robustness, and efficiency in machine learning," *arXiv preprint*, vol. arXiv:2312.14712, 2023. [Online]. Available: https://arxiv.org/abs/2312.14712

[3] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. Proceedings of Machine Learning Research, vol. 54. PMLR, 2017, pp. 1273–1282.

[4] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, "Deep learning with differential privacy," in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2016, pp. 308–318.

[5] C. Dwork, K. Kenthapadi, F. McSherry, I. Mironov, and M. Naor, "Our data, ourselves: privacy via distributed noise generation," ser. EUROCRYPT'06. Berlin, Heidelberg: Springer-Verlag, 2006, p. 486–503. [Online]. Available: https://doi.org/10.1007/11761679_29

[6] P. Blanchard, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 119–129.

[7] X. Cao, M. Fang, J. Liu, and N. Z. Gong, "FLTrust: Byzantine-robust federated learning via trust bootstrapping," in *Network and Distributed System Security Symposium (NDSS)*, 2021. [Online]. Available: https://arxiv.org/abs/2012.13995

[8] A. F. Aji and K. Heafield, "Sparse communication for distributed gradient descent," in *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2017, pp. 440–450.

[9] R. Guerraoui, N. Gupta, R. Pinot, S. Rouault, and J. Stephan, "Differential privacy and byzantine resilience in sgd: Do they add up?" 2021. [Online]. Available: https://arxiv.org/abs/2102.08166

[10] E.-M. El-Mhamdi, R. Guerraoui, A. Guirguis, L. N. Hoang, and S. Rouault, "Genuinely distributed byzantine machine learning," 2020. [Online]. Available: https://arxiv.org/abs/1905.03853

[11] M. Fang, X. Cao, J. Jia, and N. Z. Gong, "Local model poisoning attacks to Byzantine-robust federated learning," in *Proceedings of the 29th USENIX Security Symposium*, 2020, pp. 1605–1622.

[12] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, J. Fernandez-Marques, Y. Gao, L. Sani, K. H. Li, T. Parcollet, P. P. de Gusmão, and N. D. Lane, "Flower: A friendly federated learning research framework," *arXiv preprint*, vol. arXiv:2007.14390, 2020. [Online]. Available: https://arxiv.org/abs/2007.14390

[13] A. Yousefpour, I. Shilov, A. Sablayrolles, D. Testuggine, K. Prasad, M. Malek, J. Nguyen, S. Ghosh, A. Bharadwaj, J. Zhao, G. Cormode, and I. Mironov, "Opacus: User-friendly differential privacy library in PyTorch," *arXiv preprint*, vol. arXiv:2109.12298, 2021. [Online]. Available: https://arxiv.org/abs/2109.12298

[14] S. Kiani, N. Kulkarni, A. Dziedzic, S. Draper, and F. Boenisch, "Differentially private federated learning with time-adaptive privacy spending," 2025. [Online]. Available: https://arxiv.org/abs/2502.18706

[15] X.-Y. Liu, R. Zhu, D. Zha, J. Gao, S. Zhong, M. White, and M. Qiu, "Differentially private low-rank adaptation of large language model using federated learning," *ACM Transactions on Management Information Systems*, vol. 16, 2025.

10 APPENDIX

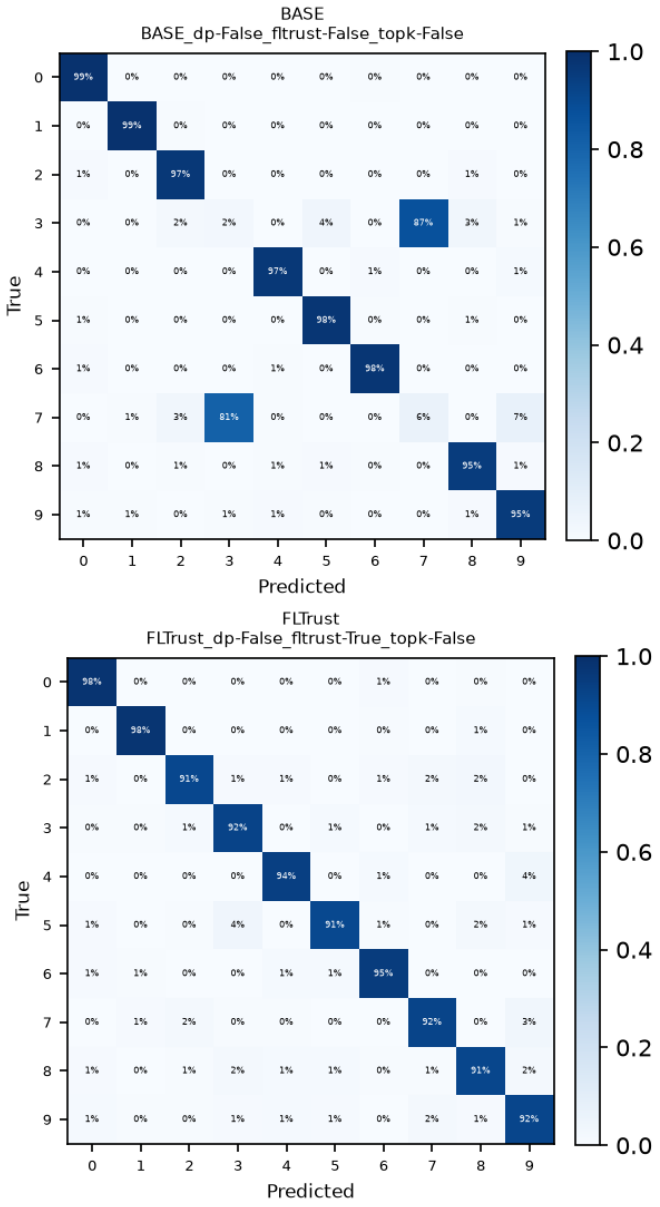

Figure 3: Final confusion matrices on the MNIST dataset for standard FedAvg (top) and with FLTrust enabled (bottom).

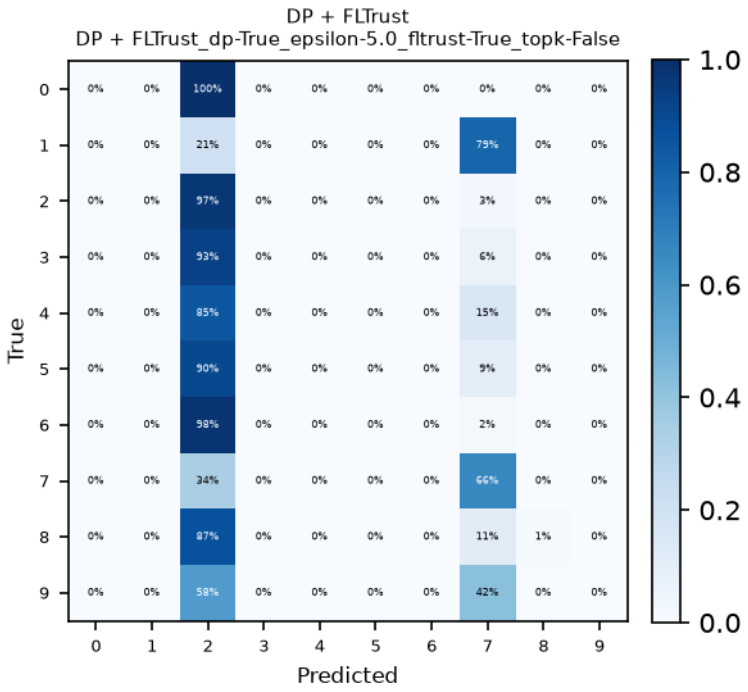

Figure 4: Final confusion matrix on the MNIST dataset for the DP+FLTrust configuration.

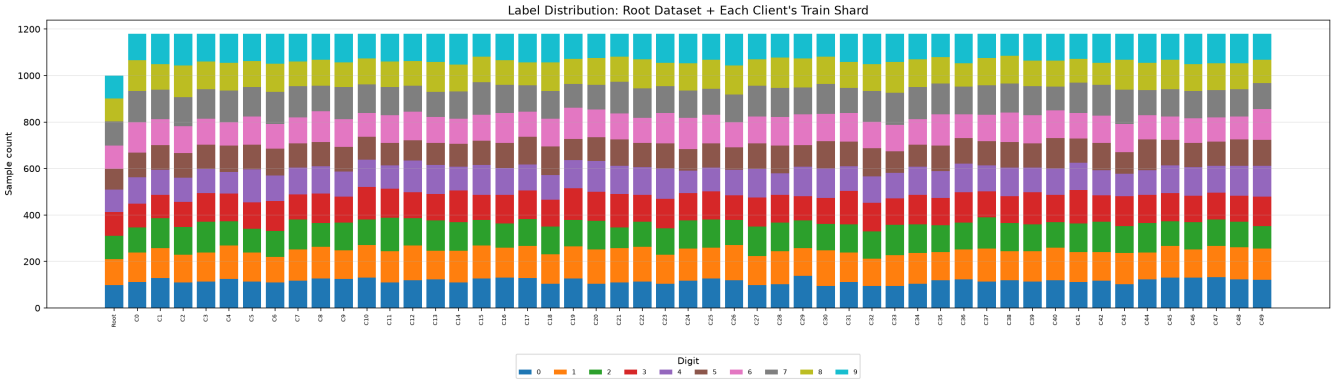

Figure 5: Label distribution of the root dataset used for FLTrust and the client training datasets. The root dataset was sampled uniformly across all classes, while the client datasets were sampled randomly.

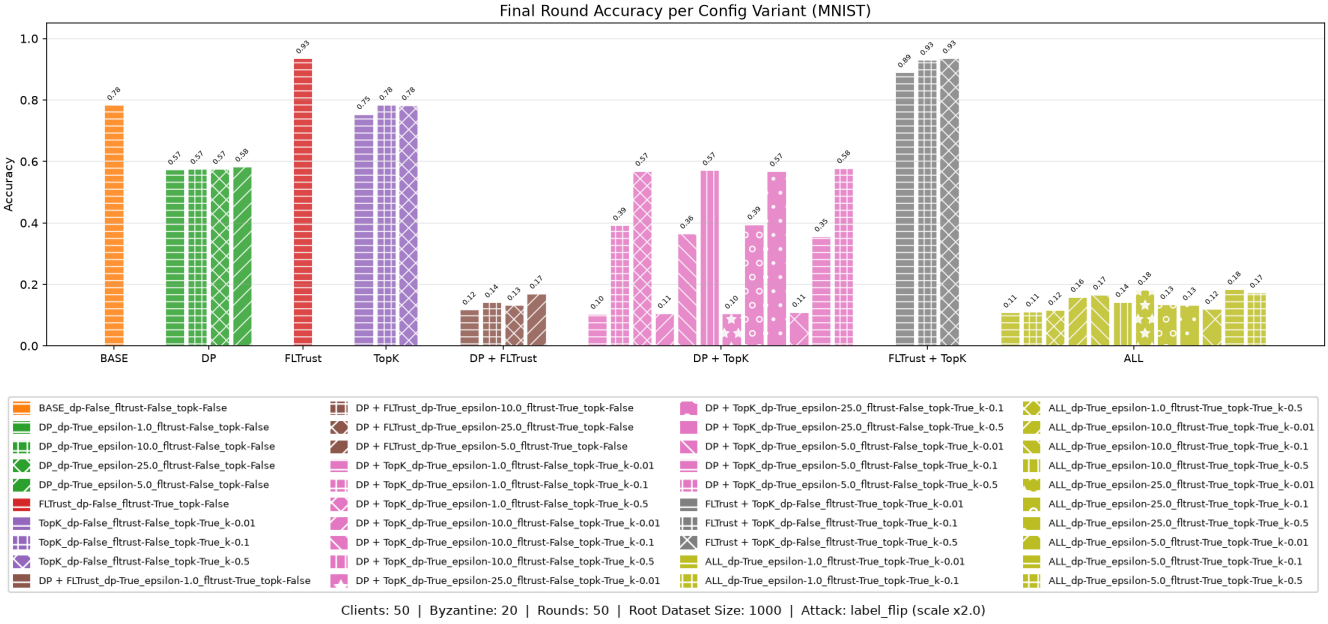

Figure 6: Final accuracy achieved after 50 rounds on all different configurations training on 50 clients at one epoch per round.

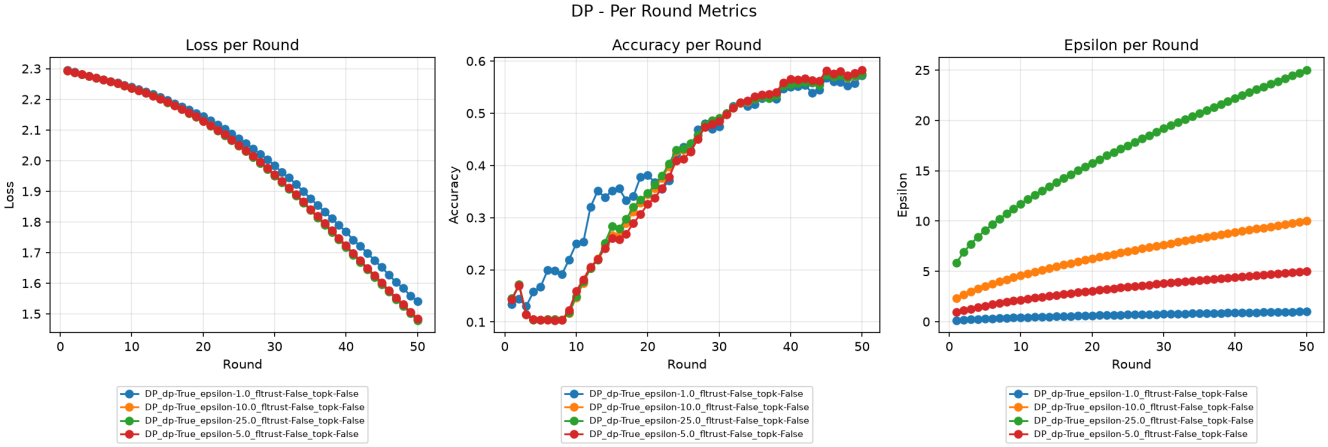

Figure 7: Accuracy per training round for DP-only configuration.

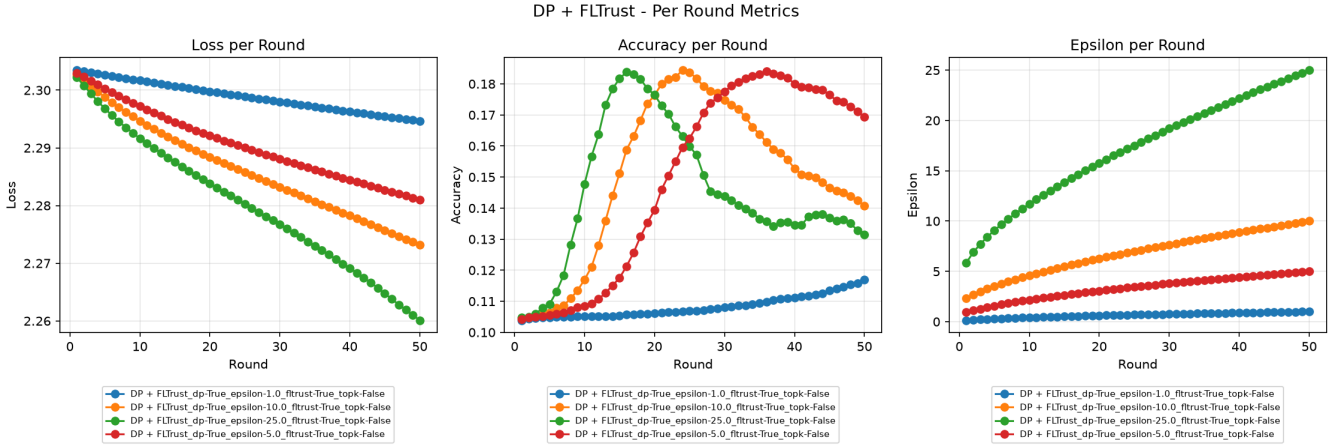

Figure 8: Accuracy per training round for DP+FLTrust configuration.

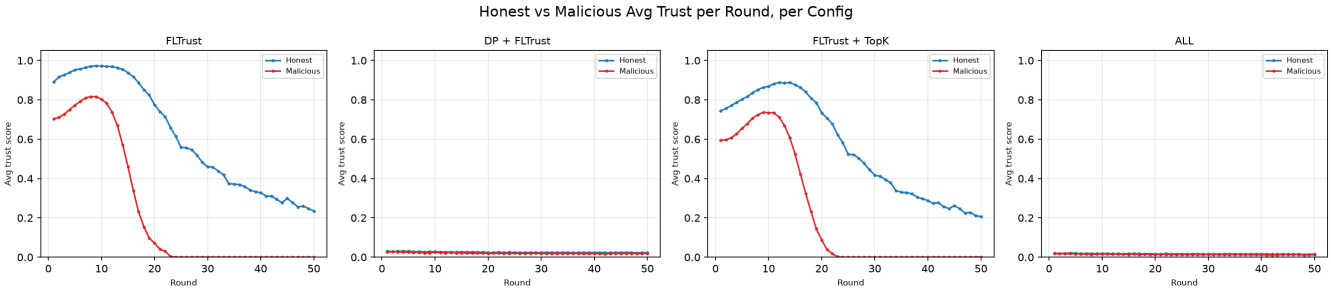

Figure 9: The line charts depict the average trust scores of honest and malicious clients during the training rounds. While for settings where DP is enabled the scores are consistently equally low, the settings without DP-related noise addition show a clear distinction.